

Bluetooth Master Mode Experiment

Bluetooth Master Mode Experiment

- 1. BLE Central Overview
- 2. Core Concepts
 - 2.1 BLE Central Communication Flow
 - 2.2 IRQ Event Flow
 - 2.3 Core API (Central-specific)
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code Explained
 - 5.1 Scanning and Matching the Target
 - 5.2 Discovering the Service and Characteristics
 - 5.3 Enabling Notify (Writing the CCCD)
 - 5.4 Receiving Notify + Sending Write
 - 5.5 Main Loop
- 6. Operating Procedures
 - 6.1 Dual-Board Debugging
 - 6.2 Test Method 1: Dual-Board Test (paired with another board flashed with the BLE Slave example)
 - 6.3 Verification Methods
- 7. Common Problems

1. BLE Central Overview

BLE (Bluetooth Low Energy) is a short-range wireless communication protocol designed for low-power, low-bandwidth scenarios. Compared with Classic Bluetooth, BLE has a faster connection speed (millisecond-level) and lower power consumption (coin-cell batteries can run for months).

The importance of the BLE central tutorial: The BLE central (Central) plays the role of actively discovering and connecting to peripherals; it is the controlling end for data aggregation and command delivery. Center devices such as phones, tablets, and gateways all run as the central — mastering central programming means being able to build a center node that actively scans, automatically connects, and reads/writes data on remote peripherals; it is the other half of the core capability for building a complete BLE system.

In this experiment the ESP32-S3 acts as a BLE central, scanning for and connecting to a peripheral (such as another ESP32-S3 running the BLE slave program), sending data via Write and receiving data via Notify.

BLE role system:

Role	GAP Name	Responsibilities
Central (Master)	Central	Actively scan nearby devices, initiate connections, discover services and characteristics, read/write data
Peripheral (Slave)	Peripheral	Advertise its own presence, accept connections, provide GATT services for the central to access

Typical application scenarios:

Application Type	Example
Data collector	Central polls sensor readings from multiple peripherals
Gateway device	BLE central collects data and uploads it to the cloud via WiFi
Central control panel	Central connects to multiple BLE peripherals for unified management

2. Core Concepts

2.1 BLE Central Communication Flow

```
[1] Scan          -> discover the target device
[2] Connect       -> establish a GATT connection
[3] Discover Services -> find the Service (UUID: 0xFFFF0)
[4] Discover Characteristics -> find TX (0xFFFF1) + RX (0xFFFF2)
[5] Write CCCD (0x0001) -> enable TX Notify notifications
[6] Bidirectional communication -> write RX (0xFFFF2) to send data, listen to TX
Notify to receive data
```

2.2 IRQ Event Flow

```
_IRQ_SCAN_RESULT          found a device, check name match
_IRQ_CENTRAL_CONNECT      connection successful, start Service discovery
_IRQ_GATTC_SERVICE_RESULT found the target Service
_IRQ_GATTC_SERVICE_DONE   Service discovery complete, start Characteristic
discovery
_IRQ_GATTC_CHARACTERISTIC_RESULT found the TX/RX characteristics
_IRQ_GATTC_CHARACTERISTIC_DONE characteristic discovery complete -> READY
_IRQ_GATTC_NOTIFY         received Notify data from the peripheral
_IRQ_CENTRAL_DISCONNECT   disconnected, rescan
```

2.3 Core API (Central-specific)

```
ble.gap_scan(0, 30000, 30000)          # start scanning
ble.gap_connect(addr_type, addr)        # connect to device
ble.gattc_discover_services(conn_handle) # discover services
ble.gattc_discover_characteristics(conn, start, end) # discover characteristics
ble.gattc_write(conn, handle, data, mode) # write characteristic (0=no
response, 1=with response)
```

3. Hardware Dependencies

Two ESP32-S3 boards are needed: one runs `ble_master.py` (central), the other runs `ble_slave.py` (peripheral; see the Bluetooth Slave Mode experiment). Just place the two boards together; the BLE communication range is about 10~30 meters.

4. Project Architecture and Flow

```
Script execution (single-file .py)
|
|- BLEMaster class initialization:
|   |- ble.active(True) + irq callback registration
|   |- gap_scan(0, 30000, 30000) // start scanning
|
|- while True:
|   |- wait for connection readiness (_ready == True)
|   |- when ready, send data: gattc_write(RX, "HELLO FROM MASTER")
|   |- sleep(2500ms)

IRQ callback (event-driven state machine):
SCAN -> CONNECT -> DISCOVER_SVC -> DISCOVER_CHR -> READY
NOTIFY received -> print the data sent by the peripheral
DISCONNECT -> rescan
```

5. Source Code Explained

Full source code: [Source Code -> MicroPython -> 4.Network Protocols -> BLE_Master -> ble_master.py](#)

5.1 Scanning and Matching the Target

`gap_scan` starts scanning; the IRQ callback parses the device name in the advertising packet, and after matching `TARGET`, stops scanning and initiates a connection.

```
TARGET = "ESP-IDF" # target device name

def _irq(self, event, data):
    if event == _IRQ_SCAN_RESULT:
        addr_type, addr, _, rssi, adv_data = data
        name = _name(adv_data) # parse the device name in
                                the advertising packet
        if name == TARGET: # name match
            print(f">>> Connecting to {TARGET}...")
            self._ble.gap_scan(None) # stop scanning
            self._ble.gattc_connect(addr_type, bytes(addr)) # initiate connection
```

5.2 Discovering the Service and Characteristics

After a successful connection, it sequentially discovers Service -> Characteristic, matching UUIDs to obtain the TX (0xFFFF1) and RX (0xFFFF2) handles.

```
elif event == _IRQ_CENTRAL_CONNECT:
    self._conn, _, _ = data
    self._ble.gattc_discover_services(self._conn) # discover all services

elif event == _IRQ_GATTTC_SERVICE_RESULT:
    _, start, end, uuid = data
    if _u16(uuid) == _SVC_UUID: # match the target service
        self._svc_start, self._svc_end = start, end
```

```

elif event == _IRQ_GATT_CHARACTERISTIC_RESULT:
    _, end, val, props, uuid = data
    u = _u16(uuid)
    if u == _CHR_TX_UUID:                                # TX (Notify) handle
        self._tx, self._tx_end = val, end
    elif u == _CHR_RX_UUID:                              # RX (write) handle
        self._rx = val

```

5.3 Enabling Notify (Writing the CCCD)

Writes 0x0001 to the TX characteristic's CCCD descriptor (UUID 0x2902) to enable the peripheral's Notify push.

```

elif event == _IRQ_GATT_CHARACTERISTIC_DONE:
    if self._tx and self._rx:
        self._ready = True
        self._ble.gattc_discover_descriptors(            # find the CCCD descriptor
            self._conn, self._tx, self._tx_end)

elif event == _IRQ_GATT_DESCRIPTOR_RESULT:
    _, dsc, uuid = data
    if _u16(uuid) == _CCCD_UUID:                        # CCCD UUID = 0x2902
        self._ble.gattc_write(self._conn, dsc, b'\x01\x00', 1) # write 0x0001
                                                                # to enable Notify

```

5.4 Receiving Notify + Sending Write

The `GATT_NOTIFY` event receives data pushed by the peripheral; `send()` writes data to the peripheral's RX characteristic to achieve bidirectional communication.

```

elif event == _IRQ_GATT_NOTIFY:
    _, _, nd = data
    print(f"<<< {bytes(nd).decode()}")                  # print the data sent by
                                                                the peripheral

def send(self, data):
    if self._ready and self._rx:
        self._ble.gattc_write(self._conn, self._rx, data, 0) # write the RX
                                                                characteristic (no response)

```

5.5 Main Loop

After ready, send "HELLO FROM MASTER" to the peripheral every 2.5s.

```

m = BLEMaster()
while True:
    if m.is_ready():
        m.send(b"HELLO FROM MASTER")                    # send to the peripheral
                                                                every 2.5s
    time.sleep_ms(2500)

```

6. Operating Procedures

6.1 Dual-Board Debugging

Thonny IDE steps: See `MicroPython -> 1. Before Development -> 3. Thonny IDE`.

6.2 Test Method 1: Dual-Board Test (paired with another board flashed with the BLE Slave example)

1. Power on the slave first and run `ble_slave.py`

```
===== ESP32-S3 BLE Slave =====
[BLE] Init ...
[BLE] Init OK, TX=16 RX=19
Ready. Waiting for connection...
[BLE] Advertising as 'ESP-IDF' ...
```

2. Run `ble_master.py` on the central

```
===== ESP32-S3 BLE Master =====
[BLE] Init ...
[BLE] Init OK, starting scan...
Scanning for 'ESP-IDF'...
Ready. Waiting for slave...
Found: midea (rssi=-93)
Found: ESP-IDF (rssi=-22)
>>> Connecting to ESP-IDF...
Connected (handle=1)
  SVC 0xFFFF0 [14..65535]
  TX(notify) handle=16
<<< Hello from ESP32
  RX(write) handle=19
  *** READY ***
  CCCD -> write 0x0001
```

3. Serial output example

Central (`ble_master.py`):

```
===== ESP32-S3 BLE Master =====
[BLE] Init ...
[BLE] Init OK, starting scan...
Scanning for 'ESP-IDF'...
Ready. Waiting for slave...
Found: midea (rssi=-93)
Found: ESP-IDF (rssi=-22)
>>> Connecting to ESP-IDF...
Connected (handle=1)
  SVC 0xFFFF0 [14..65535]
  TX(notify) handle=16
<<< Hello from ESP32
  RX(write) handle=19
  *** READY ***
  CCCD -> write 0x0001
<<< Hello from ESP32
<<< Hello from ESP32
<<< Hello from ESP32
>>> HELLO FROM MASTER
```

Peripheral (`ble_slave.py`):

```
===== ESP32-S3 BLE Slave =====
[BLE] Init ...
```

```
[BLE] Init OK, TX=16 RX=19
Ready. Waiting for connection...
[BLE] Advertising as 'ESP-IDF' ...
=====
BLE Connected
Sent: Hello from ESP32
Sent: Hello from ESP32
Sent: Hello from ESP32
=====
Recv: HELLO FROM MASTER
Sent: Hello from ESP32
Sent: Hello from ESP32
```

```
===== ESP32-S3 BLE Slave =====
[BLE] Init ...
[BLE] Init OK, TX=16 RX=19
Ready. Waiting for connection...
[BLE] Advertising as 'ESP-IDF' ...
=====
BLE Connected
Sent: Hello from ESP32
Sent: Hello from ESP32
Sent: Hello from ESP32
=====
Recv: HELLO FROM MASTER
Sent: Hello from ESP32
Sent: Hello from ESP32
Sent: Hello from ESP32
Sent: Hello from ESP32
Sent: Hello from ESP32
=====
Recv: HELLO FROM MASTER
Sent: Hello from ESP32
Sent: Hello from ESP32
```

6.3 Verification Methods

Operation	Expected Result
Run the slave first, then the central	The central automatically scans and connects
Power off the slave	The central prints <code>Disconnected</code> and rescans
Every 2.5s on the central	The peripheral serial prints <code>Recv: HELLO FROM MASTER</code>

When the slave is powered off:

```
>>> HELLO FROM MASTER
Disconnected
```

7. Common Problems

Symptom	Cause	Solution
The central cannot find the peripheral	The peripheral is not advertising or the name does not match	Confirm the peripheral is running and the <code>TARGET</code> variable matches the peripheral name
Stuck in the discovery stage after connecting	Service UUID mismatch	The UUIDs on both the central and peripheral must be the same (0xFFF0, 0xFFF1, 0xFFF2)
Cannot receive Notify data	CCCD not written correctly	Check whether <code>_IRQ_GATTC_DESCRIPTOR_RESULT</code> executes
<code>_u16()</code> parsing fails	Differences in the NimBLE UUID object format	Use <code>repr(uuid)</code> regex extraction or <code>int(str(uuid), 16)</code>